CASE STUDY 2

ORDER LIFECYCLE AND STATE MANAGEMENT

Q1: Illegal or Contradictory States in the Naive Order Class

The naive Order class uses a status string, three independent boolean values, nullable timestamps and a public list. Because these fields can be changed separately, the object can enter invalid or contradictory states.

1. Shipped order with no lines: Violates Invariant 1 because an order must contain at least one line before it is placed.
2. Cancelled order later shipped: Violates Invariant 4 because a cancelled order cannot move to the Shipped state.
3. DRAFT order with shippedAt date: Violates Invariant 5 because shipment details must exist if and only if the order has been shipped.
4. Total does not match the order lines: Violates Invariant 2 because the total must always equal the sum of all line totals.
5. Lines changed after placement: Violates Invariant 3 because lines can be added or removed only while the order is in the Draft state.
6. Duplicate product lines: Violates Invariant 6 because each product should have one line, and adding the same product should increase its quantity.

Therefore, the naive model does not protect the order's invariants and allows different parts of the program to create invalid order states.

Q2: Order State Transition Table and Ship Method

An order follows a fixed lifecycle. Only the transitions defined by the business rules are allowed.

Current State	Allowed Next State
Draft	Placed or Cancelled
Placed	Paid or Cancelled
Paid	Shipped or Cancelled
Shipped	Delivered
Delivered	No further transition specified
Cancelled	No further transition specified

The ship method in Model A is:

```
void ship(TrackingNumber trackingNumber, Instant shippedAt) {
    requireStatus(Status.PAID);

    shipment = Optional.of(
```

```
            new Shipment(trackingNumber, shippedAt)
        );

        status = Status.SHIPPED;
    }
```

Explanation:

- The method first checks that the current status is PAID.
- If the order is not PAID, requireStatus throws a domain exception and the method stops.
- A cancelled order has status CANCELLED, so it cannot pass the PAID status check.
- Shipment information is created only after the guard succeeds, and the status is then changed to SHIPPED.

This guard prevents invalid transitions, including shipping an order that has already been cancelled.

Q3: Comparison of Model A and Model B

Model A uses one Order entity with a guarded state machine. Model B uses a separate type for each order state.

Criterion	Model A – Guarded Entity	Model B – State Types
Type safety	Invalid transitions are blocked at runtime using checks.	Invalid transitions can be prevented at compile time by the available methods.
Code volume	Lower; one main Order class manages the states.	Higher; separate classes are required for different states.
State-specific data	May use Optional fields, such as Optional<Shipment>.	Data exists only in the state type that needs it.
Persistence	Simpler to store and map.	More complex; requires state discrimination and mapping for each type.

Recommendation:

For a system with eight states that changes frequently, Model A is the better choice. It keeps the code simpler, reduces the number of state-specific classes and makes frequent changes easier to manage. Its transition rules are checked at runtime through guard methods. Model B provides stronger compile-time safety, but it introduces more classes and can make frequent changes and persistence mapping more complicated.

Hence, Model A is recommended when the system has many states, frequent changes and relatively simple state-specific requirements.

Q4: Why Should Total Be Computed from Order Lines?

The total should be computed from the order lines instead of being stored as a separate field because:

- It always reflects the current prices and quantities of the order lines.
- It avoids a mismatch between the stored total and the actual line values.
- It removes the need to update the total every time a line is added, removed or changed.
- It reduces the possibility of bugs caused by forgotten or incorrect updates.

Storing the total may be justified when the system needs a fixed historical order total for invoices, reports or performance reasons. In that case, an additional invariant must be protected: the stored total must equal the sum of the order-line totals at the time the total is finalized. The lines and total should be updated together in one consistent operation.

Q5: Customer Credit Limit and Cross-Aggregate Rules

The rule that an order must not be placed if it exceeds the customer's credit limit is not an Order invariant. The Order aggregate can protect its own state and lines, but the customer's credit information belongs to a separate Customer or Credit aggregate.

The rule should be handled by a domain service or an application-layer workflow. Before placing the order, the workflow should check the customer's available credit and reserve the required amount. The order should be placed only after the credit reservation succeeds.

To avoid a race condition when two orders are submitted at the same time:

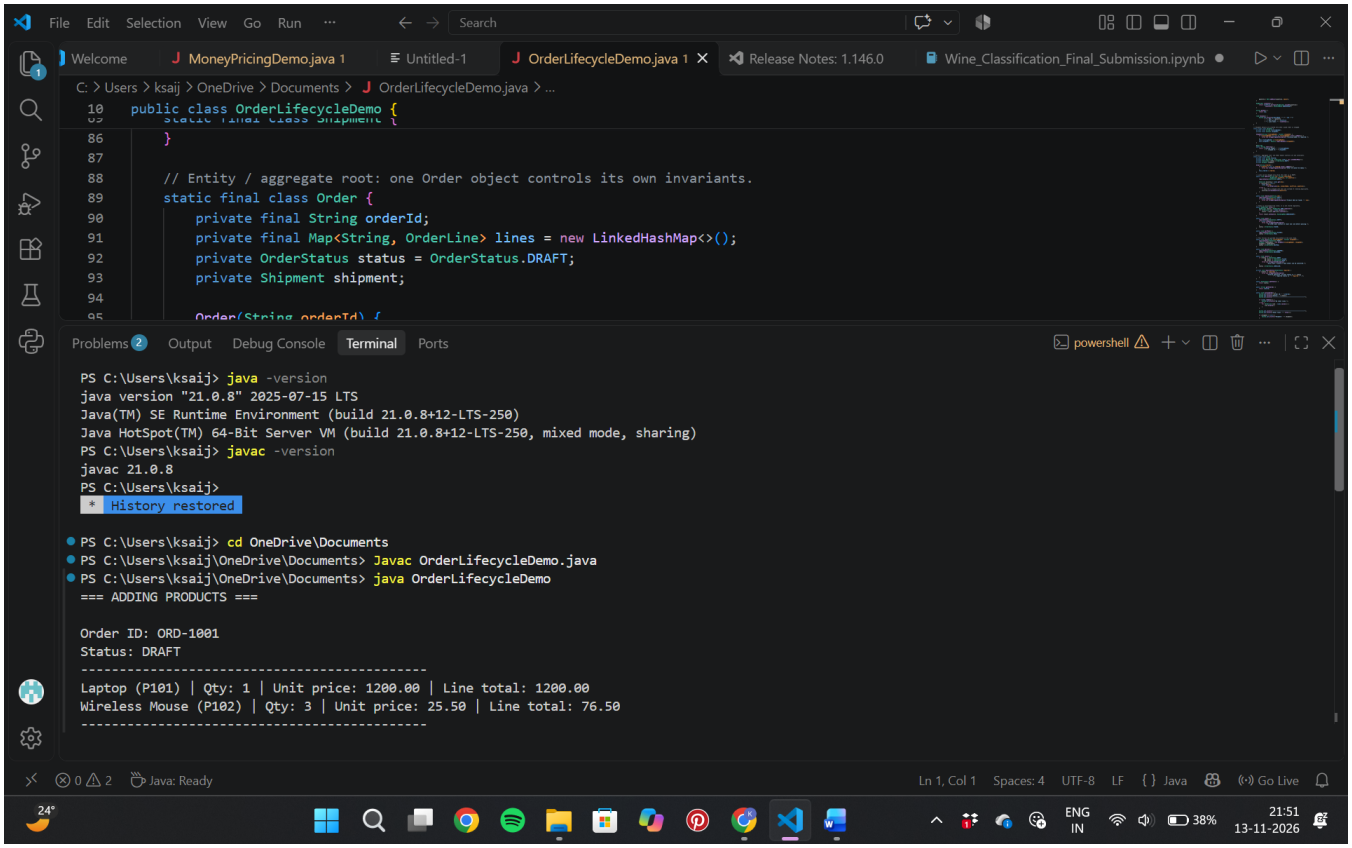
- Perform the credit check and reservation as one atomic operation.
- Use a database transaction, locking or optimistic version checking so concurrent requests cannot both spend the same available credit.
- If the reservation or order placement fails, release the reservation or use a compensation action.

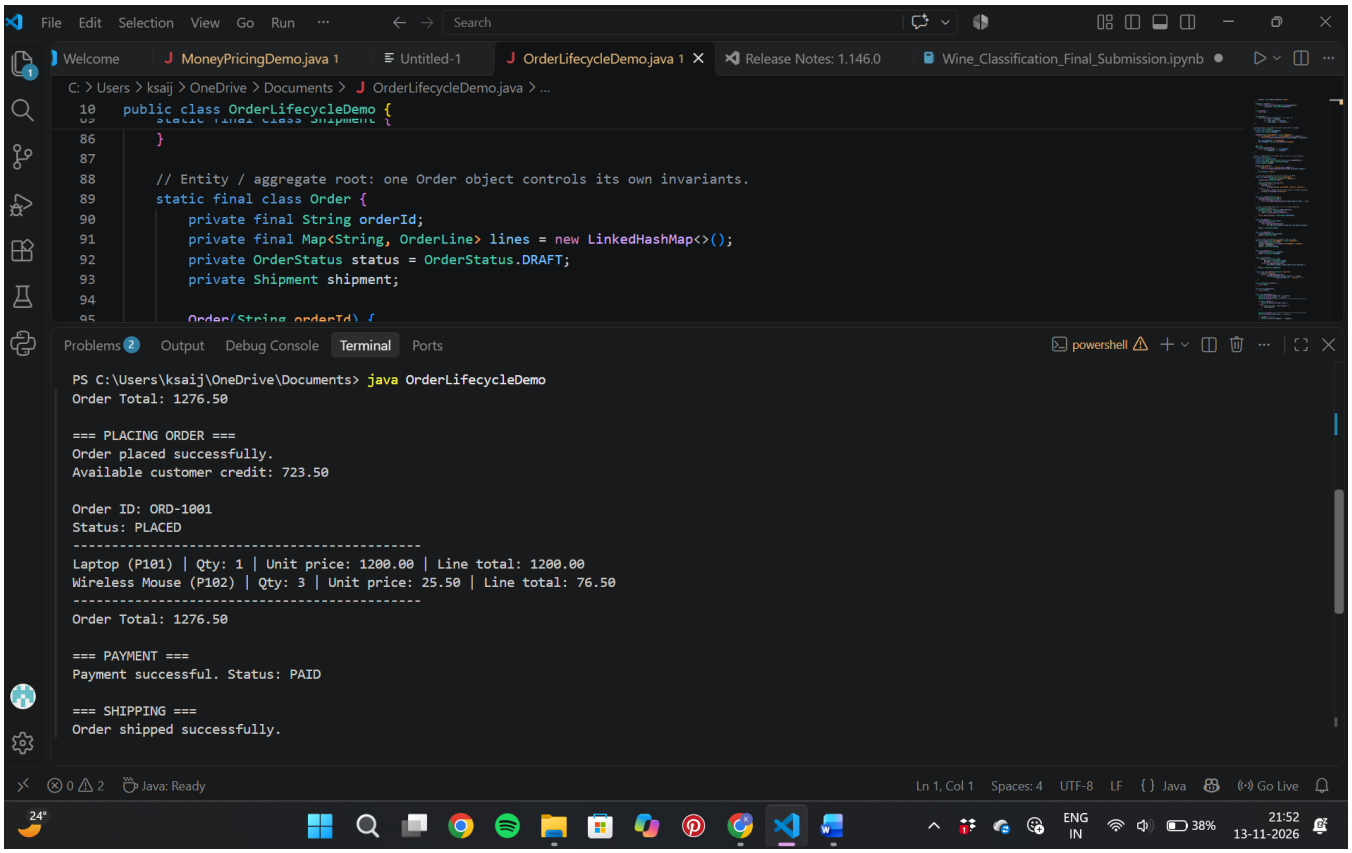
This keeps the Order aggregate focused on its own invariants while the cross-aggregate credit rule is coordinated by a domain service or application workflow.

Conclusion

This case study demonstrates how object-oriented modelling can protect an order's lifecycle and prevent invalid states. By using controlled state transitions, private data, derived totals and clear aggregate boundaries, the system becomes more reliable and easier to maintain. Model A provides a practical approach when the order states change frequently, while cross-aggregate rules such as stock and credit checks are handled by domain services or application workflows.

Output:


File Edit Selection View Go Run
Search
Welcome
MoneyPricingDemo.java 1
Untitled-1
OrderLifecycleDemo.java 1
Release Notes: 1.146.0
Wine_Classification_Final_Submission.ipynb
C: > Users > ksaij > OneDrive > Documents > OrderLifecycleDemo.java > ...
10 public class OrderLifecycleDemo {
86 }
87
88 // Entity / aggregate root: one Order object controls its own invariants.
89 static final class Order {
90 private final String orderId;
91 private final Map<String, OrderLine> lines = new LinkedHashMap<>();
92 private OrderStatus status = OrderStatus.DRAFT;
93 private Shipment shipment;
94
Problems 2
Output
Debug Console
Terminal
Ports
powershell
PS C:\Users\ksaij> java -version
java version "21.0.8" 2025-07-15 LTS
Java(TM) SE Runtime Environment (build 21.0.8+12-LTS-250)
Java HotSpot(TM) 64-Bit Server VM (build 21.0.8+12-LTS-250, mixed mode, sharing)
PS C:\Users\ksaij> javac -version
javac 21.0.8
PS C:\Users\ksaij>
* History restored
PS C:\Users\ksaij> cd OneDrive\Documents
PS C:\Users\ksaij\OneDrive\Documents> Javac OrderLifecycleDemo.java
PS C:\Users\ksaij\OneDrive\Documents> java OrderLifecycleDemo
=== ADDING PRODUCTS ===
Order ID: ORD-1001
Status: DRAFT
Laptop (P101) | Qty: 1 | Unit price: 1200.00 | Line total: 1200.00
Wireless Mouse (P102) | Qty: 3 | Unit price: 25.50 | Line total: 76.50
Java: Ready
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Java
Go Live
ENG
IN
38%
21:51
13-11-2026


File Edit Selection View Go Run
Search
Welcome
MoneyPricingDemo.java 1
Untitled-1
OrderLifecycleDemo.java 1
Release Notes: 1.146.0
Wine_Classification_Final_Submission.ipynb
C: > Users > ksaij > OneDrive > Documents > OrderLifecycleDemo.java > ...
public class OrderLifecycleDemo {
}
// Entity / aggregate root: one Order object controls its own invariants.
static final class Order {
private final String orderId;
private final Map<String, OrderLine> lines = new LinkedHashMap<>();
private OrderStatus status = OrderStatus.DRAFT;
private Shipment shipment;
Problems 2
Output
Debug Console
Terminal
Ports
powershell
PS C:\Users\ksaij\OneDrive\Documents> java OrderLifecycleDemo
Order Total: 1276.50
=== PLACING ORDER ===
Order placed successfully.
Available customer credit: 723.50
Order ID: ORD-1001
Status: PLACED
Laptop (P101) | Qty: 1 | Unit price: 1200.00 | Line total: 1200.00
Wireless Mouse (P102) | Qty: 3 | Unit price: 25.50 | Line total: 76.50
Order Total: 1276.50
=== PAYMENT ===
Payment successful. Status: PAID
=== SHIPPING ===
Order shipped successfully.
Java: Ready
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Java
Go Live
ENG
IN
38%
21:52
13-11-2026